

TypeScript – Jan 29 2020– Teachers Notes.(Level 1)

(The comprehensive material I was looking forward when I wanted to learn TypeScript)

Level 1

Learn TypeScript in UseCases.

Caution : This notes will be understandable only if you attend my sessions.

TypeScript is a relatively new language. It was launched for public use in October 2012, as the version 0.8 of the language. It was a result of two years of research at Microsoft, with Anders Hejlsberg, the lead architect of c#, as well as the creator of Delphi and Turbo Pascal.

- TypeScript is a typed superset of JavaScript that compiles to plain JavaScript.
- TypeScript is pure object oriented with classes, interfaces and statically typed like C# or Java.
- The popular JavaScript framework Angular 2.0 is written in TypeScript.
- Mastering TypeScript can help programmers to write object-oriented programs and have them compiled to JavaScript, both on server side and client side.

TypeScript – Keywords

Keywords have a special meaning in the context of a language table lists some keywords in TypeScript.

break	as	any
case	if	throw
var	number	string
module	type	instanceof
public	private	enum
finally	for	while
null	super	this
in	return	true
any	extends	static

Data Hiding

public

A public data member has universal accessibility. Data members in a class are public by default.

2.private

Private data members are accessible only within the class that defines these members. If an external class member tries to access a private member, the compiler throws an error.

3.protected

A protected data member is accessible by the members within the same class as that of the former and also by the members of the child classes.

UseCase 1 – HelloWorld

```
class Greeting{
  greet():void{
    console.log("hello and welcome to Typescript ....and Angular");
  }
}
var obj=new Greeting();
obj.greet(); obj.testIfMethod();
```

* Before we go for the next step ensure that type script is installed using nodejs – c:\>npm install –g typescript

1. Right click on file and open command prompt
2. tsc helloworld.ts
3. node helloworld.js

or use <https://www.typescriptlang.org/play/>

Ex 2 – Constructor

```
class A{
  constructor() {
    console.log("cons called...");
  }
}
new A();
```

Ex 3 – Static Key word

The static Keyword

The static keyword can be applied to the data members of a class. A static variable retains its values till the program finishes execution. Static members are referenced by the class name.

Example

```
class StaticMem {
  static num:number;
  static disp():void {
    console.log("The value of num is"+ StaticMem.num)
  }
}

StaticMem.num = 12 // initialize the static variable
StaticMem.disp() // invoke the static method
```

UseCase 2 – Basic Programming Constructs

Ex- 1 - If Else If

```
class Greeting{
    greet():void{
        console.log("hello and welcome to Typescript ....and Angular");
    }

    testIfMethod(): void{
        var num:number = 2
        if(num > 0) {
            console.log(num+" is positive")
        }
        else if (num < 0) {
            console.log(num+" is negative")
        }
        else {
            console.log(num+" is neither positive nor negative")
        }
    }
}

var obj=new Greeting();
obj.greet(); obj.testIfMethod();
```

Ex 2- Switch Case

```
class Greeting{
    greet():void{
        console.log("hello and welcome to Typescript ....and Angular");
    }
    testMethod(): void {
        var grade: string = "A";
        switch (grade) {
            case "A": {
                console.log("Excellent");
                break;
            }
            case "B": {
                console.log("Good");
                break;
            }
            case "C": {
                console.log("Fair");
                break;
            }
            case "D": {
                console.log("Poor");
                break;
            }
            default: {
                console.log("Invalid choice");
            }
        }
    }
}
```

```
        break;
    }
}
}

var obj=new Greeting();
obj.greet(); obj.testMethod();
```

Ex 3 - While Loop

```
class Greeting{
    greet():void{
        console.log("hello and welcome to Typescript ....and Angular");
    }
    testMethod(): void {
        var i:number = 1
        while(i<=10) {
            if (i % 5 == 0) {
                console.log ("The first multiple of 5 between 1 and 10 is : "+i)
                break //exit the loop if the first multiple is found
            }
            i++
        } //outputs 5 and exits the loop
    }
}

var obj=new Greeting();
obj.greet(); obj.testMethod();
```

Ex 4 - Continue KeyWord

```
class Greeting{
    greet():void{
        console.log("hello and welcome to Typescript ....and Angular");
    }
    testMethod(): void {
        var num:number = 0
        var count:number = 0;
        for(num=0;num<=20;num++) {
            if (num % 2==0) {
                continue
            }
            count++
        }
        console.log (" The count of odd values between 0 and 20 is: "+count) //outputs 10
    }
}

var obj=new Greeting();
```

```
obj.greet(); obj.testMethod();
```

UseCase 3 – Function Demo

```
class Greeting {
    static greet(): void { //static function
        console.log("hello and welcome to Typescript ....and Angular");
    }
    test() { // function definition
        console.log("function called")
    }

    //function defined
    greetWithReturn(): string { //the function returns a string
        return "Hello World"
    }

    caller() {
        Greeting.greet() //static function greet() invoked
    }

    test_param(n1: number, s1: string) {
        console.log(n1)
        console.log(s1)
    }
}

var obj=new Greeting();
obj.test_param(123, "this is a string");
obj.caller();
console.log(obj.greetWithReturn());
obj.test();
```

UseCase 4 – Arrays

```
class Greeting {
    test() {

        var alphas;
        alphas = ["1", "2", "3", "4"]; console.log(alphas[0]);
        console.log(alphas[1]);

        var arr_names:number[] = new Array(4)
        for (var i = 0; i < arr_names.length; i++)
        {
            arr_names[i] = i * 2;
            console.log(arr_names[i]);
        }
        var j: any;
        var nums: number[] = [1001, 1002, 1003, 1004];
        for (j in nums)
        {
```

```
        console.log(nums[j]);
    }
}
}
var obj=new Greeting();
obj.test();
```

Ex 2 - Function - More Experiments on Array

```
class Greeting {
    static test():string[]
    {
        return new Array("Mary", "Tom", "Jack", "Jill");
    }
    test2(){
        var nums: string[] = Greeting.test();
        for (var i in nums) {
            console.log(nums[i])
        }
        Greeting.disp(nums);
    }
    static disp(arr_names:string[]){
        for (var i = 0; i < arr_names.length; i++)
        {
            console.log(arr_names[i]);
        }
    }
}
var obj=new Greeting();
obj.test2()
```

Experiments to Do on Arrays

Array Methods

1. `var alpha = ["a", "b", "c"]; var numeric = [1, 2, 3]; var alphaNumeric = alpha.concat(numeric); console.log("alphaNumeric : " + alphaNumeric );`
2. `function isBigEnough(element, index, array) { return (element >= 10); } var passed = [12, 5, 8, 130, 44].every(isBigEnough); console.log("Test Value : " + passed );`
3. `function isBigEnough(element, index, array) { return (element >= 10); } var passed = [12, 5, 8, 130, 44].filter(isBigEnough); console.log("Test Value : " + passed );`
4. `let num = [7, 8, 9]; num.forEach(function (value) { console.log(value); });`
5. `var index = [12, 5, 8, 130, 44].indexOf(8); console.log("index is : " + index );`
6. `var arr = new Array("First","Second","Third"); var str = arr.join(); console.log("str : " + str ); var str = arr.join(", "); console.log("str : " + str ); var str = arr.join(" + "); console.log("str : " + str );`
7. `var index = [12, 5, 8, 130, 44].lastIndexOf(8); console.log("index is : " + index );`

```
8. var numbers = [1, 4, 9]; var roots = numbers.map(Math.sqrt); console.log("roots is : " + roots );

9. var numbers = [1, 4, 9]; var element = numbers.pop(); console.log("element is : " + element ); var
   element = numbers.pop(); console.log("element is : " + element );

10. var numbers = new Array(1, 4, 9); var length = numbers.push(10); console.log("new numbers is : " +
    numbers ); length = numbers.push(20); console.log("new numbers is : " + numbers );

11. var total = [0, 1, 2, 3].reduce(function(a, b){ return a + b; }); console.log("total is : " + total );

12. var total = [0, 1, 2, 3].reduceRight(function(a, b){ return a + b; }); console.log("total is : " + total
    );

13. var arr = [0, 1, 2, 3].reverse(); console.log("Reversed array is : " + arr );

14. var arr = [10, 1, 2, 3].shift(); console.log("Shifted value is : " + arr );

15. var arr = ["orange", "mango", "banana", "sugar", "tea"]; console.log("arr.slice( 1, 2) : " + arr.slice(
    1, 2) ); console.log("arr.slice( 1, 3) : " + arr.slice( 1, 3) );

16. function isBigEnough(element, index, array) { return (element >= 10); } var retval = [2, 5, 8, 1,
    4].some(isBigEnough); console.log("Returned value is : " + retval );

17. var arr = new Array("orange", "mango", "banana", "sugar"); var sorted = arr.sort();
    console.log("Returned string is : " + sorted );

18. var arr = new Array("orange", "mango", "banana", "sugar"); var sorted = arr.sort();
    console.log("Returned string is : " + sorted );
```

UseCase 5 –TypeScript Tuples

At times, there might be a need to store a collection of values of varied types. Arrays will not serve this purpose. TypeScript gives us a data type called tuple that helps to achieve such a purpose. It represents a heterogeneous collection of values. In other words, tuples enable storing multiple fields of different types. Tuples can also be passed as parameters to functions.

```
var mytuple = [10,"Hello"]; //you can create a tuple by just passing different
types of data in a single array.
console.log(mytuple[0])
console.log(mytuple[1])
```

Ex 1 – Basic Array operations using tuples

```
class Greeting {
    test(): void {
var mytuple = [10,"Hello","World","typeScript"];
console.log("Items before push "+mytuple.length)    // returns the tuple size

mytuple.push(12)                                     // append value to the tuple
console.log("Items after push "+mytuple.length)
console.log("Items before pop "+mytuple.length)
console.log(mytuple.pop()+" popped from the tuple") // removes and returns the last item

        console.log("Items after pop " + mytuple.length)
            //update a tuple element
            mytuple[0] = 121
            console.log("Tuple value at index 0 changed to  "+mytuple[0])
    }
}

var obj=new Greeting();
obj.test();
```

UseCase 6 – Interfaces

```
interface ILoan {
interest: number;
sayHi():void;
sayHello(): string;
}

class AgriLoan implements ILoan {
interest:number
rebate:number
constructor(interest:number,rebate:number) {
this.interest = interest;
this.rebate = rebate;
}
sayHi() {
console.log("hello");
}
sayHello(): string{
return "hello";
}
}

var obj = new AgriLoan(10,1)
console.log("Interest is : " + obj.interest + " Rebate is : " + obj.rebate)
obj.sayHi();
console.log(obj.sayHello());
```

Ex 2 – More On Interfaces

```
interface IPerson {
firstName:string,
lastName:string,
```



```
sayHi: ()=>string
}
var customer:IPerson = {
  firstName:"Tom",
  lastName:"Hanks",
  sayHi: ():string =>{return "Hi there"}
}

console.log("Customer Object ")
console.log(customer.firstName)
console.log(customer.lastName)
console.log(customer.sayHi())

var employee:IPerson = {
  firstName:"Jim",
  lastName:"Blakes",
  sayHi: ():string =>{return "Hello!!!"}
}
console.log("Employee Object ")
console.log(employee.firstName) console.log(employee.lastName)
```

UseCase 7 – Inheritance

Ex1 – Interface Inheritance

```
interface Person {
  age:number
}
interface Musician extends Person {
  instrument:string
}
var drummer = <Musician>{};
drummer.age = 27
drummer.instrument = "Drums"
console.log("Age: " + drummer.age);
console.log("Instrument: " + drummer.instrument)
```

Ex 2 – Class Inheritance

```
class Shape {
  Area:number
  constructor(a:number) {
    this.Area = a
  }
}
class Circle extends Shape {
  disp():void {
    console.log("Area of the circle: "+this.Area)
  }
}
```

```
var obj = new Circle(223);  
obj.disp()
```

Ex 3 – Inheritance example on Method Overriding

```
class PrinterClass {  
  doPrint():void {  
    console.log("doPrint() from Parent called...")  
  }  
}  
  
class StringPrinter extends PrinterClass {  
  doPrint():void {  
    super.doPrint()  
    console.log("doPrint() is printing a string...")  
  }  
}  
  
var obj = new StringPrinter()  
obj.doPrint()
```

UseCase 8 – Design Patterns implementation in Typescript

Strategy Pattern

```
class PaintBrush{  
  p:Paint=new Paint();  
  setPaint(p:Paint){  
    this.p=p;  
  }  
  doPaint(){  
    this.p.display();  
  }  
}  
  
class Paint{  
  color: string="";  
  display() {  
    console.log(this.color);  
  }  
}  
  
class RedPaint extends Paint{  
  color="RED....";  
}  
  
class BluePaint extends Paint{  
  color="Blue...";  
}  
  
class GreenPaint extends Paint{  
  color="Green..";  
}
```

```
}  
  
var brush=new PaintBrush();  
var rp=new RedPaint();  
var bp=new BluePaint()  
brush.setPaint(bp);  
  
brush.doPaint();
```

UseCase 9 – Design Pattern

Ex – Above Strategy Pattern example using generics

```
class Brush<T> {  
    input: T;  
    constructor(input: T) {  
        this.input = input;  
    }  
    getInput() {  
        return this.input;  
    }  
}  
  
class Paint { doPaint(): void { console.log("painting");}}  
class Water { doSprinkle(): void { console.log("sprinkling"); }}  
let paint = new Paint();  
let water = new Water();  
let waterbrush = new Brush<Water>(water);  
let paintbrush = new Brush<Paint>(paint);  
  
waterbrush.getInput().doSprinkle();  
paintbrush.getInput().doPaint();
```

UseCase 10 – Strategy Design Pattern – One more example

```
class Fan{  
    s: State=new SwitchOffState();  
    pull() {  
        this.s.pull(this);  
    }  
}  
  
class State{  
    pull(f:Fan): void { }  
}  
class SwitchOffState extends State{  
    pull(f:Fan) {  
        console.log("Switch on state.....");  
        f.s = new SwitchOnState();  
    }  
}
```

```
    }  
  }  
  class SwitchOnState extends State{  
    pull(f:Fan) {  
      console.log("Low speed state.....");  
      f.s = new LowSpeedState();  
    }  
  }  
  class LowSpeedState extends State{  
    pull(f:Fan) {  
      console.log("High speed state.....");  
      f.s = new HighSpeedState();  
    }  
  }  
  class HighSpeedState extends State{  
    pull(f:Fan) {  
      console.log("High speed state.....");  
      f.s = new SwitchOffState();  
    }  
  }  
}  
  
var fan = new Fan();  
for (var i = 0; i < 5; i++){  
  fan.pull();  
}
```

UseCase 11 – Exception Exercise

```
class Handler911 {  
  handle1(dbr:DogBarkException):void{  
    console.log("dog is running...let us chase and play...");  
  }  
  handle2(dbe:DogBiteException):void{  
    console.log("dog has bitten,,will take him to hospital...");  
  }  
  handle3(dhe:DogHappyException):void{  
    console.log("dog is happy,,,let us play...");  
  }  
}  
  
class DogExceptions { visit(): void { } };  
class DogBiteException{visit():void  
  {new Handler911().handle1(this);}};  
class DogBarkException{visit():void  
  {new Handler911().handle2(this);}};  
class DogHappyException {  
  visit(): void {  
    { new Handler911().handle3(this); };  
  };  
}  
class Dog{  
  play(item:Item){  
    item.execute();  
  };  
};  
class Item { execute(): void { } }
```

```
class Stone{execute():void{throw new DogBiteException();}}
class Stick{execute():void{throw new DogBarkException();}}
class Biscuit{execute():void{throw new DogHappyException();}}
```

```
class Child{
    playWithDog(item,dog):void{
        try{
            dog.play(item);
        }catch(e){
            e.visit();
        }
    }
};

var bujji=new Child();
var jimmy=new Dog();
var stone=new Stone();
var stick=new Stick();
var biscuit=new Biscuit();
bujji.playWithDog(biscuit,jimmy);
```

*****The End*****